Taller: AGENTS.md, skills y modo Plan

BoviControl Pro

1. Herramienta y repositorios

El taller se hizo sobre BoviControl Pro, una aplicación para llevar el registro del ganado. Los planes se generaron en **OpenCode, con el agente Plan**. La revisión del código, los cambios y las pruebas se hicieron con apoyo de Codex.

En este proyecto, el backend y el frontend están en el mismo repositorio. El backend usa Flask y SQLite; la interfaz está hecha con React y TypeScript. Por eso los enlaces apuntan a dos partes del mismo proyecto.

2. Memoria del proyecto

Se escogió **AGENTS.md** porque OpenCode lo puede leer al trabajar en el proyecto. El archivo de la raíz explica cómo instalar, ejecutar y probar la aplicación en PowerShell. El archivo de src añade las indicaciones de la interfaz.

Había dos detalles que convenía dejar por escrito: los datos de cada usuario deben mantenerse separados y la pantalla guarda los cambios por **/api/state**, no por las rutas antiguas. También quedaron las reglas para las ramas y commits, y qué archivos no se deben tocar. Estos archivos llegaron a main en el PR #1.

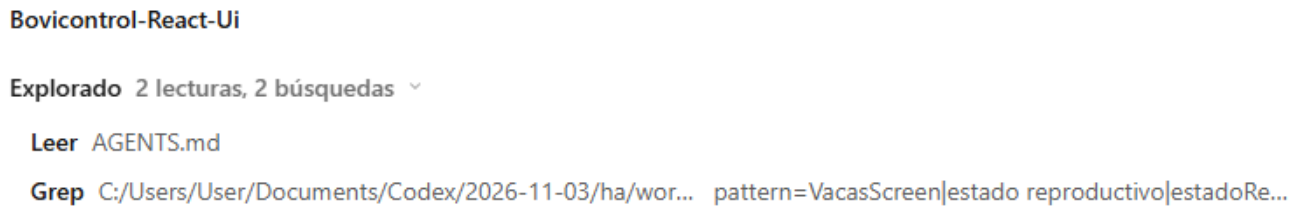
3. Skills instaladas y evidencia de uso

Se crearon dos skills para las tareas del taller. bovicontrol-api-tests sirve para probar el backend en una base temporal. bovicontrol-react-ui orienta los cambios de interfaz: filtros, etiquetas y manejo con teclado.

Ambas quedaron en **.opencode/skills/**, cada una en su propia carpeta. Se revisó su contenido completo: son instrucciones en Markdown, no traen programas ni piden contraseñas o descargas. Las dos pasaron el validador y se usaron en OpenCode al preparar los planes.



Uso de bovicontrol-api-tests para preparar la primera solicitud.



Uso de bovicontrol-react-ui para preparar el filtro de vacas.

Documentación consultada: reglas · skills · agente Plan.

4. Solicitudes

Solicitud 1. Evitar errores al guardar datos. La primera mejora salió de un fallo concreto: si el identificador de un registro llegaba como una lista, el servidor respondía con un error 500. La solicitud fue detectar ese dato incorrecto antes de guardarlo y explicar el problema sin alterar los registros anteriores.

La versión resumida del plan se entrega en **plan-1-bovicontrol.md**. La salida original de OpenCode está en las evidencias del proyecto. El cambio quedó en main mediante el PR #2.

Este fue el prompt usado en OpenCode:

"Actúa en modo Plan de OpenCode para BoviControl Pro. Como encargado de la finca quiero que PUT /api/state rechace registros malformados antes de guardar, para conservar mis datos y recibir un error comprensible en lugar de un error 500. Lee AGENTS.md, carga la skill bovicontrol-api-tests usando la herramienta skill y revisa app.py y tests/test_reproduction.py. Mantén Flask, AppState por usuario y el contrato con las cinco listas actuales. Valida que cada elemento sea objeto y tenga id de texto no vacío, sin espacios en los extremos y único dentro de su lista; permite el mismo id en listas diferentes. Rechaza identificadores numéricos, listas u objetos, y elementos nulos o escalares con HTTP 400 y error en español, sin modificar el estado guardado. No amplíes el cambio a reglas de fechas, migraciones, rutas antiguas ni configuración de producción. El plan debe indicar archivos, ubicación de la validación antes de construir diccionarios, pruebas de entradas válidas e inválidas, persistencia intacta, aislamiento entre dos usuarios y regresión reproductiva. Ejecuta solo lectura y la skill; no implementes ni delegues. Devuelve el plan final en español y Markdown, listo para guardarse como plan-1-bovicontrol.md, sin texto adicional fuera del plan. "

Qué se revisó del plan. Lo principal era que la validación ocurriera antes de intentar guardar. También se revisó que el cambio no afectara las reglas de reproducción ni los datos de otras cuentas. El plan cubría esos puntos; se añadieron comprobaciones de sesión y del token que protege las escrituras.



Cómo se comprobó el resultado. Pasaron las ocho pruebas nuevas del backend y la prueba de reproducción que ya tenía el proyecto. Se probaron datos incorrectos y dos cuentas distintas. El caso que antes daba error 500 ahora devuelve 400 con un mensaje en español, y la información guardada permanece igual. Las pruebas usaron SQLite, no MySQL.

4. Solicitudes (continuación)

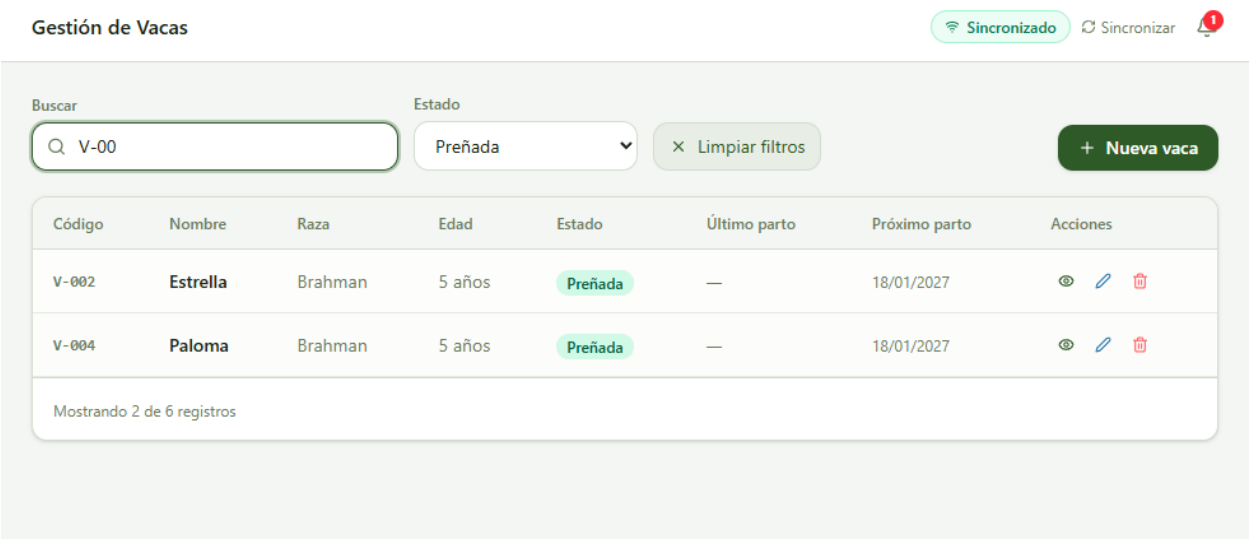
Solicitud 2. Encontrar las vacas por su estado. La pantalla ya permitía buscar por nombre o código, pero faltaba filtrar por estado reproductivo. La mejora permite, por ejemplo, ver solo las vacas preñadas y buscar dentro de ese grupo. También añade un botón para limpiar los filtros.

La versión resumida del plan se entrega en **plan-2-bovicontrol.md**. La salida original de OpenCode está en las evidencias del proyecto. El cambio quedó en main mediante el PR #3.

Este fue el prompt usado en OpenCode:

"Actúa en modo Plan de OpenCode para BoviControl Pro. Como encargado de la finca quiero filtrar la lista de vacas por estado reproductivo y combinarlo con la búsqueda por nombre o código para encontrar los animales que necesito revisar. Lee AGENTS.md y src/AGENTS.md, carga la skill bovicontrol-react-ui usando la herramienta skill y revisa VacasScreen en src/App.tsx. Agrega un selector accesible con Todos los estados, Preñada, Vacía, En celo, Lactando y Seca; combina el filtro con la búsqueda mediante AND, conserva búsqueda sin distinguir mayúsculas y recorta espacios de la consulta. Incluye Limpiar filtros, contador de resultados actualizado y estado vacío. Usa estado local y controles nativos con etiquetas visibles; filtrar no debe cambiar los registros ni enviar PUT /api/state. Conserva la arquitectura y las reglas reproductivas. No agregues dependencias ni cambies otras pantallas. El plan debe identificar archivos, pruebas reales en navegador con datos sintéticos, teclado, pantalla estrecha y compilación con npm.cmd run build incluyendo static/dist. Ejecuta solo lectura y la skill; no implementes ni delegues. Devuelve el plan final en español y Markdown, listo para guardarse como plan-2-bovicontrol.md, sin texto adicional fuera del plan."

Qué se revisó del plan. El primer plan necesitó ajustes. Se pidió una etiqueta visible para la búsqueda y una forma de desplazar la tabla en pantallas pequeñas. Después se corrigieron los pasos de teclado y algunos resultados esperados de las pruebas. OpenCode generó la versión final; las dos correcciones quedaron guardadas en docs/taller.



Cómo se comprobó el resultado. Se probaron 14 combinaciones de búsquedas y estados, el botón para limpiar, el teclado y una pantalla de 700 píxeles. Filtrar no envió solicitudes de guardado ni cambió los datos. La aplicación compiló y la prueba de reproducción siguió pasando. Solo se mantuvo el aviso de tamaño del archivo de JavaScript que ya existía.